Практика №2. Аудит готового клиент-серверного приложения

Ответы на вопросы 1)

Ответ 1. В send_reliable используется сокет SOCK_STREAM и вызов connect(). Это TCP, потому что SOCK_STREAM — потоковый сокет, он сначала устанавливает соединение (connect()), а потом передаёт данные надёжно и по порядку.

Ответ 2. В send_fast используется сокет SOCK_DGRAM и вызов sendto(). Это UDP, потому что SOCK_DGRAM — датаграммный сокет, соединение не устанавливается, данные отправляются без гарантии доставки. Поэтому в клиенте и стоит таймаут 2 секунды.

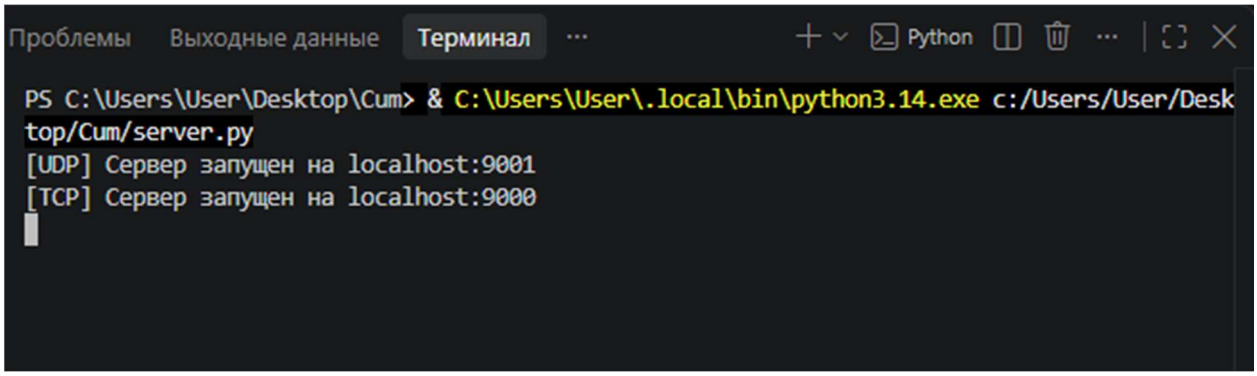
Ответ 3. Данные передаются в формате JSON. Это видно по строке в client.py:

client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))
json.dumps() превращает словарь в JSON-строку, потом она кодируется в байты и отправляется.

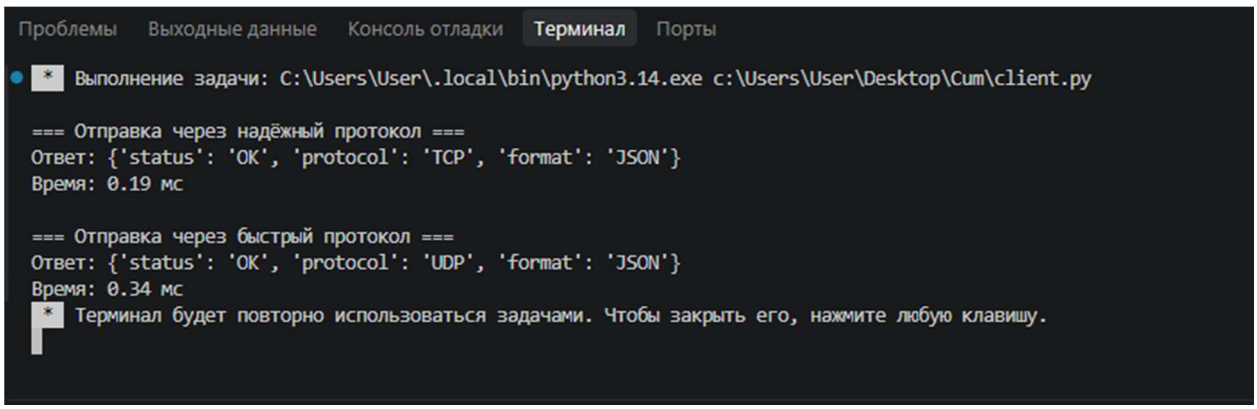
Ответ 4. Если отправить "hello", сервер попробует сделать json.loads("hello"), получит ошибку JSONDecodeError, поймает её в except, выведет в консоль [TCP] Ошибка: не удалось разобрать JSON и отправит клиенту {"status": "ERROR", "reason": "invalid JSON"}.

Скриншоты 2)

Сервак:



Клиент:



3) Тестирование:

1) Сервер выключен, клиент TCP Клиент вызывает connect(), но на порту 9000 никто не слушает. Появляется ошибка ConnectionRefusedError: [Errno 111] Connection refused, и программа падает

2) Клиент вызывает sendto(), и ошибки нет — UDP не проверяет, есть ли сервер. Данные просто уходят в никуда. Потом клиент ждёт ответ 2 секунды (таймаут), не получает его и возвращает {"status": "TIMEOUT"}

3) После замены на client.send(b"hello") сервер получает b"hello", пытается разобрать как JSON, получает JSONDecodeError. В консоли сервера: TCP] Ошибка: не удалось разобрать JSON от (...)

4) Сравнение форматов format_check.py выводит JSON и XML и их размеры. Для нашего объекта:

JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}

XML: <weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather>

JSON компактнее.

4) Вывод:

Я выбрал TCP и Json:

TCP: В коде клиент отправляет погодные данные и ожидает ответа от сервера. TCP гарантирует доставку данных в правильном порядке и подтверждает установку соединения - это важно, когда нужно быть уверенным, что запрос дошёл и ответ получен.

JSON: Компактнее XML, проще читается и естественно преобразуется в словари Python через `json.loads()` / `json.dumps()`. XML требует дополнительного разбора через ElementTree и даёт более громоздкую структуру с тегами.

Итог: TCP + JSON - надёжная доставка и компактный, удобный формат. UDP + XML имел бы смысл только там, где важна максимальная скорость и допустима потеря отдельных сообщений